

W10 — Docker Compose & Multi-Container

ML Engineer Journey | Phase 3 — Docker & Clean Architecture | Mei 2026

Overview Minggu Ini

Week 10 membangun di atas Docker fundamentals (Week 9) dengan memperkenalkan orkestrasi multi-container menggunakan Docker Compose. Kamu belajar mengelola beberapa service sekaligus — API, Redis, Postgres — dengan networking, volume persistence, environment management, healthcheck, dan integration testing. Kulminasi minggu ini adalah menjalankan seluruh Project 1 (Risk Scoring API) via satu perintah `docker-compose up`.

Ringkasan Per Hari

D01 — docker-compose.yml Fundamentals

Menulis compose file pertama untuk satu service. Memahami struktur dasar: services, ports, environment, volumes, networks. Perbedaan antara `docker-compose up` (foreground) vs `-d` (detached).

D02 — Multi-Service Compose + Networking

Menambahkan Redis sebagai service kedua. Memahami bahwa Docker Compose secara otomatis membuat internal DNS — service name menjadi hostname. API bisa reach Redis via 'redis:6379' tanpa IP hardcoded.

D03 — Volume Mounting & Data Persistence

Named volume vs bind mount. Named volume dikelola Docker dan survive container restart. Bind mount mount langsung folder dari host — cocok untuk dev (hot reload kode, akses log langsung).

D04 — Environment Variables dalam Compose

Dua mekanisme: `environment`: (inject satu-per-satu, `${VAR}` di-resolve dari host shell) vs `env_file`: (inject seluruh file ke container). Membuat `docker-compose.dev.yml` sebagai override file untuk dev workflow.

D05 — Health Checks & Readiness Probe

`depends_on`: `service_healthy` vs `service_started`. Healthcheck block: `test`, `interval`, `timeout`, `retries`, `start_period`. Exponential backoff retry untuk koneksi Redis. `GET /health` harus return 503 jika dependency gagal.

D06 — Compose untuk Project 1 + Integration Test

Menyatukan semua konsep minggu ini: `env_file`, named volume, healthcheck, override file. Debug nyata: menemukan bahwa `environment: ${VAR}` meng-override `env_file` dengan string kosong dari host shell. Stack berjalan penuh: Redis healthy → API healthy → `/health` return JSON yang benar.

Konsep Kritis yang Harus Diingat

1. environment: vs env_file: — Perbedaan Fundamental

environment: di-resolve dari *host shell* saat Docker Compose mem-parse YAML — bukan dari dalam container. Jika variabel tidak ada di shell host, hasilnya string kosong. **env_file**: membaca file dan meng-inject seluruh isinya langsung ke dalam container runtime. Untuk dev workflow yang benar, gunakan `env_file`: di service definition, bukan `--env-file` di CLI (itu hanya untuk substitusi YAML, bukan inject ke container).

Aspek	environment:	env_file:
-------	--------------	-----------

Siapa yang baca	Docker Compose (host)	Docker Engine (container)
<code>\${VAR}</code> di-resolve dari	Shell host	Tidak ada — literal value
Jika var tidak ada di host	String kosong	N/A — baca dari file
Cocok untuk	Override spesifik	Inject banyak variable
Precedence	Lebih tinggi	Lebih rendah dari environment:

2. Override File Merge Strategy

Ketika menjalankan `docker-compose -f docker-compose.yml -f docker-compose.dev.yml up`, Docker Compose melakukan deep merge — file yang lebih kanan (kanan) memenangkan key yang sama. Base file adalah fondasi, override file hanya mendefinisikan perbedaan. Tidak perlu menduplikasi seluruh konfigurasi di override file.

3. depends_on: Batas Tanggung Jawabnya

depends_on: condition: service_healthy menjamin urutan startup — container A tidak akan start sebelum healthcheck container B lulus. Tapi tugasnya selesai di situ. Setelah aplikasi running, `depends_on` tidak mengawasi koneksi ongoing. Jika konfigurasi salah (misal `REDIS_HOST=wrong_host`), `depends_on` tidak bisa mencegah kegagalan koneksi di dalam aplikasi — itulah kenapa exponential backoff retry penting.

4. Named Volume Lifecycle

Named volume memiliki lifecycle terpisah dari container. `docker-compose down` menghapus container dan network, tapi **tidak** menghapus volume. Volume baru terhapus dengan `docker-compose down -v` atau `docker volume prune`. Prinsip ini disengaja: container bisa mati, data harus abadi.

5. Healthcheck: curl -f adalah Wajib

Tanpa flag `-f`, curl selalu exit 0 meskipun server return 4xx atau 5xx. Docker healthcheck membaca exit code — exit 0 = healthy, non-zero = unhealthy. Tanpa `-f`, container akan dianggap healthy meskipun endpoint return 500.

Arsitektur Compose Stack — Project 1

Service	Image	Healthcheck	Depends On	Volume
api	build: .	<code>curl -f /health</code>	redis (healthy)	./logs, ./app/model
redis	redis:7-alpine	<code>redis-cli ping</code>	—	redis-data (named)

Kode Referensi Penting

docker-compose.yml — Base (env_file pattern yang benar)

```
services:
  api:
    build: .
    env_file:
      - .env # inject semua variable ke container
    depends_on:
      redis:
```

```

condition: service_healthy
healthcheck:
test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
interval: 30s
timeout: 10s
retries: 3
start_period: 15s

redis:
image: redis:7-alpine
env_file:
- .env
volumes:
- redis-data:/data # named volume: data persist
healthcheck:
test: ["CMD", "redis-cli", "ping"]
interval: 10s
timeout: 5s
retries: 5
start_period: 5s
volumes:
redis-data:

```

docker-compose.dev.yml — Override Dev

```

services:
api:
env_file:
- .env.dev # override: dev variables menang
volumes:
- ./app:/app/app # bind mount: hot reload
- ./app/model:/app/model
ports:
- '8000:8000'
restart: 'no' # sadar kalau crash

redis:
ports:
- '6379:6379' # expose untuk RedisInsight/debug
restart: 'no'

```

/health endpoint — Cek Redis + Model

```

@app.get('/health')
async def health_check(request: Request):
redis_client = request.app.state.redis_client
model_service = request.app.state.model_service

# Cek Redis
try:
redis_client.ping()

```

```
redis_status = 'connected'
except Exception as e:
    raise HTTPException(503, detail='Redis unavailable')

# Cek Model – jika None, prediksi tidak bisa berjalan
if model_service is None or model_service.model is None:
    raise HTTPException(503, detail='Model belum ter-load')

return {
    'status': 'healthy',
    'redis': redis_status,
    'model_version': model_service.version,
    'version': settings.app_version
}
```

Gotchas & Kesalahan Umum

■ **environment: \${VAR}** dari host shell

Jika host shell tidak punya variabel tersebut, hasilnya string kosong — bukan error, bukan nilai dari `.env.dev`. Pydantic kemudian crash saat mencoba parse string kosong sebagai bool atau int. Solusi: gunakan `env_file`: di service definition, bukan environment: dengan `${VAR}`.

■ **--env-file** di CLI $\neq$ **env_file**: di YAML

`--env-file` di CLI hanya untuk substitusi `${VAR}` di dalam YAML saat parsing. Tidak ada variable yang dikirim ke dalam container dari flag ini. `env_file`: di dalam service definition yang inject ke container.

■ **curl** tanpa **-f** selalu **exit 0**

Docker healthcheck membaca exit code. Tanpa `-f`, curl tidak peduli HTTP status code — exit 0 meskipun server return 500. Container akan dianggap healthy padahal endpoint error. Selalu gunakan `curl -f`.

■ **depends_on** tidak mengawasi **post-startup**

`depends_on`: `service_healthy` hanya mengatur urutan startup. Setelah container running, `depends_on` sudah selesai tugasnya. Konfigurasi salah (host, password, port) tetap bisa menyebabkan koneksi gagal meskipun `depends_on` terpenuhi.

■ **docker-compose down** tidak hapus **volume**

Named volume bertahan setelah down. Ini fitur, bukan bug — data harus survive restart. Gunakan `down -v` hanya saat benar-benar ingin reset data (development, testing). Di production, jangan pernah `down -v` tanpa backup.

■ **start_period** bukan **retry**

`start_period` adalah grace period murni — kegagalan healthcheck selama periode ini tidak dihitung ke retries. Ini untuk memberi waktu aplikasi selesai inisialisasi (load model, establish connection) sebelum Docker mulai menilai apakah container healthy atau tidak.

Workflow Harian — Perintah yang Harus Hafal

Tujuan	Perintah
Jalankan stack dev	<code>docker-compose -f docker-compose.yml -f docker-compose.dev.yml up --build</code>
Jalankan detached	<code>docker-compose -f docker-compose.yml -f docker-compose.dev.yml up -d</code>
Lihat status container	<code>docker-compose ps</code>
Lihat log API	<code>docker-compose logs -f api</code>
Test health endpoint	<code>curl.exe -s http://localhost:8000/health</code>
Exec ke container	<code>docker-compose exec api bash</code>
Stop semua container	<code>docker-compose down</code>
Stop + hapus volume	<code>docker-compose down -v</code>
Restart satu service	<code>docker-compose restart api</code>

Output Files Week 10

- `docker-compose.yml` — Base compose file, `env_file` pattern, healthcheck semua service
- `docker-compose.dev.yml` — Override dev: bind mount, expose ports, restart: no
- `docker-compose.prod.yml` — Override prod: restart: unless-stopped, no exposed ports
- `.env` — Production environment variables
- `.env.dev` — Development environment variables
- `app/routers/health.py` — `/health` endpoint dengan cek Redis + model
- `app/database.py` — Redis client dengan exponential backoff retry
- `W10_Week_Summary.pdf` — Dokumen ini

Next — Week 11

Week 11 masuk ke Clean Architecture & Layered Design. Kamu akan refactor seluruh Project 1 ke pattern Router → Service → Repository. Setiap layer punya tanggung jawab yang jelas dan tidak boleh bocor ke layer lain. Topik: Layered Architecture, Dependency Injection yang proper, Separation of Concerns, dan unit testing per layer dengan pytest.

github.com/lolivampire/ML-Project | W10 Complete ✓